Shaik. Imran.
192525348.

SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO4:
Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2,BL6)

Assessment Tool Weightage: 40%

Annexure A

82.1.

Coding challenge

1. Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.

2. Write a Python function named validate_segment(length_m) to verify whether a given **UTP cable segment** satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of **100 meters**. The function should return **True** for valid cable lengths and return **False** along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least **three different test cases** and interpret the results.

3. Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named csma_log.txt, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.

4. Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.

5. Design and implement a Python class named StarSwitch that models the basic functionality of an Ethernet switch. The class should include methods such as add_port(host_id) to connect a host, remove_port(host_id) to disconnect a host, and broadcast(frame) to transmit a frame to all connected ports except the source port. The program should maintain a record of active ports and log every broadcast operation, indicating the source host and the destination hosts that successfully receive the frame. Demonstrate the implementation using multiple hosts and explain the significance of broadcasting in Ethernet communication.

RUBRICS FOR CALCULATION

	Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
A	Problem Understanding	15% 1.5	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
B	Algorithm Design	20% 2	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
C	Code Quality	15% 1.5	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
D	Functionality	20% 2	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
E	Error Handling	10% 1	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
F	Efficiency	10% 1	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
G	Documentation & Comments	5% 0.5	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
H	Testing & Debugging	5% 0.5	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done

A1

	A	B	C	D	E	F	G	H	
Q1	1	1.5	1	1.	1	0.5	0.5	0.5	7
Q2	1	1.5	1.	1.5	1	0.5	0.5	0.5	7.5
Q3	1.5	2	2.5	2.	1.	1.	0.5	0.5	10
Q4	1.5	2	1.5	2	0.5	1	0.5	0.5	9.5
Q5	1	1.5	1	1.5	0.5	0.5	0.5	0.5	7

SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO4:
Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2,BL6)

Assessment Tool Weightage: 40%

Annexure A

Coding challenge

1. Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.

   Enter number of hosts: 3

   Enter Cat6 cable length for Host 1 in meters: 20
   Host Label: Host1
   Connected To: Central_Switch
   Cable Type: Cat6 UTP
   Cable Length: 20.0 meters

   Enter Cat6 cable length for Host 2 in meters: 35
   Host Label: Host2
   Connected To: Central_Switch

Cable Type: Cat6 UTP
Cable Length: 35.0 meters

Enter Cat6 cable length for Host 3 in meters: 50
Host Label: Host3
Connected To: Central_Switch
Cable Type: Cat6 UTP
Cable Length: 50.0 meters

----- TOPOLOGY -----
Host1 --------> Central_Switch
Host2 --------> Central_Switch
Host3 --------> Central_Switch

Verification:
All hosts are connected through the Central Switch.

Explanation

In a **star topology**, every host is directly connected to a central switch. If one host or cable fails, the other hosts can continue communicating. This makes the network easier to manage and troubleshoot.

2. Write a Python function named validate_segment(length_m) to verify whether a given **UTP cable segment** satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of **100 meters**. The function should return **True** for valid cable lengths and return **False** along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least **three different test cases** and interpret the results.

Python Function to Validate UTP Cable Length

The question specifies a maximum Ethernet cable segment length of **100 meters** and asks for at least three test cases.

```
def validate_segment(length_m):
    if length_m <= 0:
        return False, "Error: Cable length must be greater than 0 meters."

    elif length_m <= 100:
        return True, "Valid cable length."

    else:
        return False, "Error: Cable length exceeds the 100 meter limit."



# Test Cases

test_cases = [50, 100, 120]

for length in test_cases:
    result, message = validate_segment(length)

    print("Cable Length:", length, "meters")
    print("Result:", result)
    print("Message:", message)
    print()
```

Sample Output:

Cable Length: 50 meters
Result: True

Message: Valid cable length.

Cable Length: 100 meters
Result: True
Message: Valid cable length.

Cable Length: 120 meters
Result: False
Message: Error: Cable length exceeds the 100 meter limit.

Interpretation

- **50 meters** → Valid
- **100 meters** → Valid because it is the maximum allowed length
- **120 meters** → Invalid because it exceeds 100 meters

3. Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named csma_log.txt, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.

Python Program to Simulate CSMA/CD

This program simulates four hosts, transmission attempts, collisions, random back-off intervals, and writes the events to `csma_log.txt`.

```
import random
import time
```

```
hosts = ["Host1", "Host2", "Host3", "Host4"]

log_file = open("csma_log.txt", "w")

for attempt in range(1, 6):

    host = random.choice(hosts)

    message = f"\nTransmission Attempt {attempt}: {host} wants to send a frame."
    print(message)
    log_file.write(message + "\n")

    # Simulate collision
    collision = random.choice([True, False])

    if collision:
        message = "Collision detected!"
        print(message)
        log_file.write(message + "\n")

        # Random back-off interval
        backoff = random.randint(1, 5)

        message = f"{host} waits for {backoff} seconds before retransmission."
        print(message)
        log_file.write(message + "\n")

        time.sleep(1)
```

```
            message = f"{host} retransmits the frame successfully."
            print(message)
            log_file.write(message + "\n")

        else:
            message = f"{host} transmitted the frame successfully."
            print(message)
            log_file.write(message + "\n")

    log_file.close()

    print("\nAll events are recorded in csma_log.txt")
```

Example Output:

```
Transmission Attempt 1: Host2 wants to send a frame.
Collision detected!
Host2 waits for 3 seconds before retransmission.
Host2 retransmits the frame successfully.

Transmission Attempt 2: Host4 wants to send a frame.
Host4 transmitted the frame successfully.
```

Explanation:

CSMA/CD works as follows:

1. A host checks whether the communication medium is free.
2. If the medium is free, it starts transmission.
3. If a collision occurs, the host detects it.
4. The host waits for a random back-off time.
5. The host then tries to transmit again.

Thus, CSMA/CD helps reduce repeated collisions by making devices wait for different random time intervals before retransmitting.

4. Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.

Client-Server Application Using Python Socket

The question asks for a client and server communicating through a simulated central switch, with a MAC address table updated after successful transmission.

```
import socket


HOST = "127.0.0.1"

PORT = 5000


# Simulated MAC Address Table

mac_table = {}


server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)


server.bind((HOST, PORT))

server.listen(1)


print("Switch Server is waiting for connection...")


conn, addr = server.accept()


print("Connected by:", addr)
```

```
while True:
    data = conn.recv(1024).decode()

    if not data:
        break

    # Expected format: Source_MAC,Port,Message
    source_mac, port, message = data.split(",")

    # MAC Learning
    mac_table[source_mac] = port

    print("\nFrame Received")
    print("Source MAC:", source_mac)
    print("Port:", port)
    print("Message:", message)

    print("\nUpdated MAC Address Table:")

    for mac, port_number in mac_table.items():
        print(mac, "-> Port", port_number)

    conn.send("Frame received and MAC table updated.".encode())
```

```
conn.close()

server.close()
```

Client Program:

```
import socket

HOST = "127.0.0.1"
PORT = 5000

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

client.connect((HOST, PORT))

source_mac = "AA:BB:CC:DD:EE:01"
port = "1"
message = "Hello from Host1"

frame = source_mac + "," + port + "," + message

client.send(frame.encode())

response = client.recv(1024).decode()

print("Server Response:", response)

client.close()
```

Sample Output

Server

```
Switch Server is waiting for connection...

Connected by: ('127.0.0.1', 54321)

Frame Received
Source MAC: AA:BB:CC:DD:EE:01
Port: 1
Message: Hello from Host1

Updated MAC Address Table:
AA:BB:CC:DD:EE:01 -> Port 1
```

5. Design and implement a Python class named StarSwitch that models the basic functionality of an Ethernet switch. The class should include methods such as add_port(host_id) to connect a host, remove_port(host_id) to disconnect a host, and broadcast(frame) to transmit a frame to all connected ports except the source port. The program should maintain a record of active ports and log every broadcast operation, indicating the source host and the destination hosts that successfully receive the frame. Demonstrate the implementation using multiple hosts and explain the significance of broadcasting in Ethernet communication.

Python Class **`StarSwitch`**

The question requires methods to add and remove ports and broadcast a frame to every connected host except the source host.

```
class StarSwitch:


    def __init__(self):

        self.active_ports = []


    # Connect a host

    def add_port(self, host_id):


        if host_id not in self.active_ports:

            self.active_ports.append(host_id)

            print(host_id, "connected to the switch.")


        else:
```

```
        print(host_id, "is already connected.")

# Disconnect a host
def remove_port(self, host_id):

    if host_id in self.active_ports:
        self.active_ports.remove(host_id)
        print(host_id, "disconnected from the switch.")

    else:
        print(host_id, "is not connected.")

# Broadcast a frame
def broadcast(self, source_host, frame):

    if source_host not in self.active_ports:
        print("Error: Source host is not connected.")
        return

    print("\nBroadcast Operation")
    print("Source Host:", source_host)
    print("Frame:", frame)

    destinations = []
```

```
        for host in self.active_ports:

            if host != source_host:
                print("Frame sent to:", host)
                destinations.append(host)

        print("Broadcast completed.")
        print("Destination Hosts:", destinations)


# Create Switch Object
switch = StarSwitch()

# Add Hosts
switch.add_port("Host1")
switch.add_port("Host2")
switch.add_port("Host3")
switch.add_port("Host4")

# Broadcast Frame
switch.broadcast("Host1", "Hello Network")

# Remove a Host
switch.remove_port("Host4")
```

```
# Broadcast Again

switch.broadcast("Host2", "Second Broadcast Message")
```

Sample Output:

```
Host1 connected to the switch.
Host2 connected to the switch.
Host3 connected to the switch.
Host4 connected to the switch.

Broadcast Operation
Source Host: Host1
Frame: Hello Network
Frame sent to: Host2
Frame sent to: Host3
Frame sent to: Host4
Broadcast completed.
Destination Hosts: ['Host2', 'Host3', 'Host4']

Host4 disconnected from the switch.

Broadcast Operation
Source Host: Host2
Frame: Second Broadcast Message
Frame sent to: Host1
Frame sent to: Host3
Broadcast completed.
Destination Hosts: ['Host1', 'Host3']
```

Explanation

Broadcasting means sending a frame to **all connected hosts except the source host**.

It is useful when:

- The destination MAC address is unknown.
- A network device needs to send information to all devices.
- Network discovery or specific broadcast communication is required.

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done